

## PART 1 OF 5. CHOOSE ONE TASK AND CHECK WHAT YOU CAN ACCESS.

# Find the thing worth automating

Open your sent messages and find a question you've asked more than once. Maybe it's "who owns this?" or "is that date still right?" Pick one you could answer, at least in part, by looking at a work tracker.

I'm a senior technical program manager at Microsoft, and I've built tools to help with status follow-up. This series walks you through making a small version for your own team. You'll start with a task brief, then write the rule, review the output, and test messages with people who've agreed to take part.

You don't need to install anything for this part. You'll need a few examples of the task, permission to use the relevant data, and somewhere to write down your decisions. If access isn't approved yet, use made-up records.

## 1 Find a task you can describe with a real example

Look through a recent week of your own work. Write down the repeated question, where you looked for the answer, and what you did with it. If the task doesn't happen every week, use a period that includes it.

Choose something small enough to review yourself. "Show me open items past their due date" gives you a list to check. "Manage the team's delivery" leaves too many decisions unspecified.

### EXAMPLE

A fictional task note: "Before our Tuesday meeting, I open the tracker, filter for open items past their due date, and ask each owner whether the record needs an update." The first version could prepare that list without contacting anyone.

Copy this into your notes and fill it out:

- ☐ The question I repeat:
- ☐ When I ask it:
- ☐ Where I look for the answer:
- ☐ The steps I take now:
- ☐ What the first version should produce:
- ☐ What I'll still decide myself:

**WHY THIS MATTERS:** a project that starts from the tool produces something impressive that saves twenty minutes a month. Starting from a question you already type keeps the first version small enough to finish.

## 2 Check whether building it would help

Count how often the task happens and measure how long a normal run takes. Include checking the result, because automation won't remove that work at the start. Compare the possible savings with setup and maintenance time.

### EXAMPLE

Fictional estimate: a task takes 15 minutes, three times a week. That's 45 minutes a week today. If checking the automated result takes 10 minutes a week, the possible saving is 35 minutes. A four-hour setup would take about seven weeks to recover, before allowing for maintenance.

Then separate the facts you can collect from the decisions you need to make. A date in the past tells you the tracker is overdue. It doesn't tell you whether the work finished, the date changed elsewhere, or the team needs help.

Check that the source contains enough information to produce the first output. If the answer only exists in a conversation, you might prepare a question for the owner instead. Don't label an item "on track" because its due date is still in the future.

**WHY THIS MATTERS:** the setup time is real and the checking time doesn't go away. If you skip the estimate you find out in week six, after you've already built it.

## 3 Ask for specific access before connecting a tool

Find out who manages the tracker and which access method your company approves. That might be an export, an existing report, or an API, which lets a program read records from the system. Don't assume your normal account can create an integration.

### EXAMPLE

"Hey, I'd like to test a read-only report for our team's open work. I need item IDs and links, titles, owners, states, and due dates. What's the approved way to read those fields? The first version won't send messages or change records. Where can I store the output, and can our approved code assistant work with this data?"

Read-only access still exposes the records you read. Agree on the project scope, who can see the output, and when saved files should be deleted. Don't paste company records, private documentation, or credentials into an unapproved service.

If approval takes time, continue with fictional items. You can work out the rules without copying a colleague's data.

**WHY THIS MATTERS:** people stall here for weeks waiting to feel ready to ask. It's usually one message to whoever manages the tracker, and you can keep working with fictional records while you wait for the answer.

## 4 Match each required value to a field in your tracker

Open a few records and find the fields below. Record their actual names, which can differ between teams even when they use the same product.

- ☐ Source and item ID: a stable identifier that distinguishes this item from another project or tracker with the same number.
- ☐ Item link and title: enough information for you to open and recognize it.
- ☐ Owner: the assigned account's ID, not just the person's display name.
- ☐ State: the team's known active and completed states.
- ☐ Due date: the agreed field and whether it stores a date or a timestamp.
- ☐ State history, if needed: a reliable time for the last state change.

Two people can share a display name, so messaging later also needs a verified match between the tracker account and the messaging account. If that match isn't available, keep the item for review rather than guessing the recipient.

**EXAMPLE**

An item called "Update setup instructions" has an owner and a state but no due date. Record "due date missing." Don't give it today's date to make the report work.

For a rule about how long a state has stayed unchanged, a general "last modified" field isn't enough. A comment or title edit may update that field. Part 2 explains what you can report when state history is unavailable.

**WHY THIS MATTERS:** most of these projects fail at the data, not the code. Ten minutes confirming the fields exist saves you from building a rule on a field that turns out to be empty on half your items.

## 5 Finish the brief you'll use to build the first version

Write the scope before asking an assistant to write code. Include the project, the kinds of items to include, the fields you've found, and the output you want. List what is still unknown so the assistant doesn't fill in those choices.

### EXAMPLE

"Prepare a local preview of follow-up messages for the team's work items. Use fictional records first. Show the item ID, owner, due date, state, and reason for including each item. Put missing or conflicting information in a separate review list. Don't send messages or change any tracker records."

I used Claude Code for my automation work. Use whichever development tools your company approves. Before Part 3, check their installation instructions and arrange help from someone who can review the generated code if you can't assess it yourself.

Keep credentials out of the prompt and source code. When you're ready to connect the tracker, use your company's approved sign-in or secret-storage method and limit access to the agreed scope.

**WHY THIS MATTERS:** a code assistant fills in any decision you leave out, and it won't tell you which ones it filled in. Listing the unknowns is how those choices stay on your desk instead of ending up buried in the script.

Before moving on, fill in the task note, estimate the time involved, and list the fields and access you have. Mark anything still waiting on approval.

Part 2 turns that brief into rules you can check against individual records.